

04 — Edge and SPDE Roadmap

What you'll learn: the staged path beyond the first build — (1) the $O(n)$ edge port of the velocity GP, and (2) the physics-informed ocean-current SPDE field — with scope, effort, and deps.

Prereqs: references/05, 06, 07; 02-gp-velocity-module-design.

Why it matters here: the eventual target is on-vehicle, and this is where the PDE-duality work lands.

Stage 0 (now) — GPyTorch velocity GP, demo-first

Covered by docs 00-03. Accuracy-focused, ground-side, torch-based. Deliberately not edge-deployable; everything below removes that limitation or extends capability.

Stage 1 — $O(n)$ edge port of the velocity GP

Goal: run the same DVL-velocity GP on the vehicle with no GPU/torch, at $O(n)$ cost.

How: swap the engine inside `GpDvlVelocityModel` for the **state-space / Kalman** form (references/05). A Matérn-3/2 temporal GP = a 2-state-per-axis linear SDE = a tiny Kalman filter/RTS smoother. The `predict()/ingest()` API and all pipeline wiring are **unchanged** (02 §7).

Reuse `imu-drift`: take the process-noise parameters from the crate's **Allan-variance** fit (`imu-drift/src/allan.rs`) and closed-form variances (`imu-drift/src/processes.rs`, e.g. `VelocityRandomWalk`, `GaussMarkov`) so the prior is grounded in the vehicle's real IMU characterization rather than learned hyperparameters (references/05 §5).

Hyperparameters: learn them once **off-board** with the Stage-0 GPyTorch model on logged data, then **freeze** them in the on-board recursion (references/08 §2f).

Effort: moderate. Pure numpy (or extend the `no_std` Rust crate). Validate by checking the state-space posterior matches the batch GP on the Stage-0 benchmark (they should agree to numerical precision for Matérn kernels), then re-run the RMSE comparison.

Deps: none new (drop torch). Optionally Rust if going fully embedded.

Stage 2 — Physics-informed ocean-current SPDE field

Goal: model the **current field** the vehicle moves through, to bound drift even without DVL bottom-lock, using the PDE duality.

How (the duality in action,

references/06-07):

1. Represent the current field as a Gaussian field defined by the **advection-diffusion SPDE**

$\partial X / \partial t + (\gamma \cdot \nabla) X - \nabla \cdot (D \nabla X) + \kappa^2 X = ?$ — γ = drift/current, D = diffusion (references/07 §2).

2. Discretize by **FEM** on a mesh of the operating area → a **GMRF** with **sparse precision** (references/06 §3); add **streamline-diffusion** stabilization where advection dominates.

3. Impose **boundary conditions** — Neumann "no-flow" at the seabed, Dirichlet at known inflows

(references/07 §3); optionally a

divergence-free kernel for incompressibility (references/07 §4).

4. Keep **Markov-in-time** (Kalman) over the GMRF for an efficient spatio-temporal filter (references/05 §7, references/08 §2c).

Use in navigation: the field provides a **current prior**; combined with water-relative velocity it reconstructs ground velocity during bottom-lock loss, bounding drift. It becomes a new measurement

source / process input to the EKF (a larger integration than Stage 0-1).

Effort: large. New subsystem (meshing, FEM assembly, sparse solves), a way to estimate γ/D (from data or an ocean model), and validation data. This is a research-grade extension, not a quick follow-up.

Deps: `scipy.sparse` (or a FEM library) for sparse assembly/solves; meshing (fmesher-style or a Python equivalent). Heavier than the rest; justified only when the current field is the priority.

Staging summary

Stage	Capability	Cost/where	New deps	Effort
0 (now)	velocity GP: denoise / adaptive R / bridge	$O(n^3)$, small n ; ground-side	<code>gpytorch</code> , <code>torch</code> (opt-in)	small
1	same GP, on-vehicle	$O(n^2)$; edge	none (drop <code>torch</code>)	moderate
2	physics-informed current field	$O(n) - O(n^{3/2})$; ground or edge	<code>scipy.sparse</code> / FEM / mesh	large

Recommended sequencing

1. Ship Stage 0; prove the benefit on the benchmark.
2. Do Stage 1 once the velocity GP earns its place — it's the real edge deliverable and reuses

`imu-drift`.

3. Treat Stage 2 as a separate research initiative when current-driven drift (not DVL dropout) becomes

the dominant error and survey data is available to fit/validate the field.

Key takeaways

- **Stage 1** = swap the velocity GP's engine to the **$O(n)$ state-space form** (same API, no pipeline

change), seed it from ``imu-drift`` Allan-variance params, freeze hyperparameters learned off-board → true edge deployment, no `torch`.

- **Stage 2** = the full **PDE-duality** payoff: an **advection-diffusion SPDE/GMRF** current field

with **boundary conditions**, bounding drift without bottom-lock — large, research-grade, separate.

- Sequence them; don't block the high-value velocity GP on the ambitious field model.

Further reading

- Back to references/06 and references/07 for the SPDE math.
- references/09 for all sources.